Models for Count Data — Poisson Regression(a special case of Generalized Linear Model)

Simon Leung 14 min read · Feb 10, 2025

Experimenting with Overdispersion and Underdispersion in Count Data

Image from https://depositphotos.com/

Poisson Regression is a powerful statistical tool for modeling count data or event data across various fields such as epidemiology, finance, and social sciences. It is used for data representing occurrences over a fixed interval, such as website visits, customer arrivals, or disease cases. Poisson Regression is specifically designed for *non-negative integer* outcomes. It models the relationship between predictors (independent variables) and the expected count of events, under the assumption that the event count follows a *Poisson distribution*.

Model Assumptions:

- The dependent variable (*non-negative integers*) follows a Poisson distribution.
- The mean and variance of the dependent variable *are equal* (can be relaxed using quasi-Poisson or negative binomial models).
- *Log-linear* relationship between predictors and the expected count.

Generalized linear modeling is a framework for statistical analysis that includes linear, logistic, Poisson, and Binomial regression as special cases.

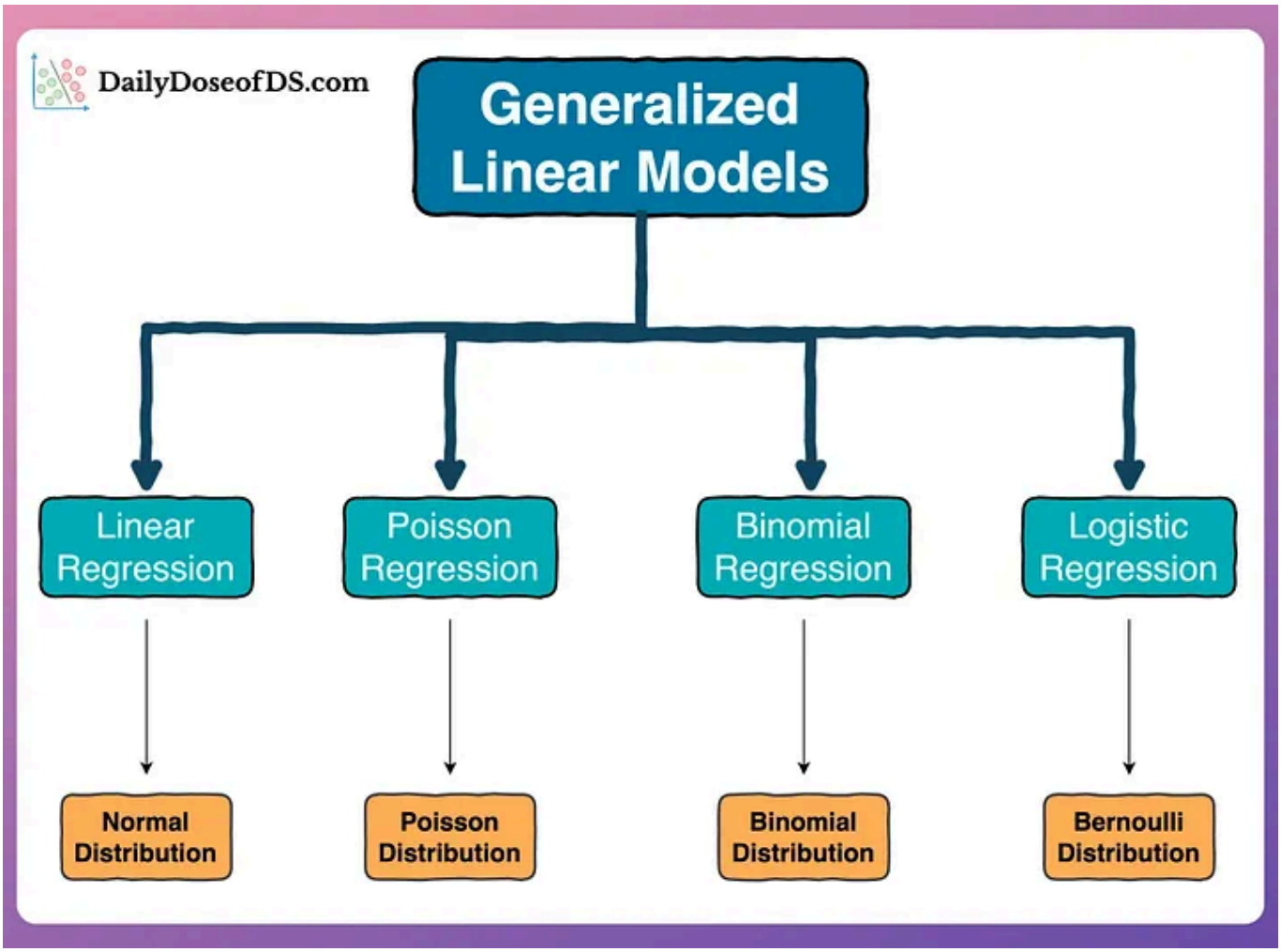


Image from [1]

Aspect	Linear Regression	Logistic Regression	Poisson Regression	Binomial Regression
Normality of Residuals	Required	Not required	Not required	Not required
Multicollinearity	Sensitive	Sensitive	Sensitive	Sensitive
Autocorrelation	Assumes none	Assumes independence	Assumes independence	Assumes independence
Correlation of Variables	Problematic	Problematic	Problematic	Problematic
Homoscedasticity	Required	Not required	Not required	Not required
Outliers	Highly sensitive	Moderately sensitive	Sensitive	Sensitive

Overdispersion

Overdispersion occurs when the variance of the count data is *greater than the mean*. This is a common issue in count data modeling. Here are some strategies to address overdispersion:

Negative Binomial Regression:

- The negative binomial regression model is a generalization of the Poisson regression model that includes an additional parameter to account for overdispersion. It allows the variance to be greater than the mean.

Quasi-Poisson Regression:

- Quasi-Poisson regression adjusts the standard errors of the estimates to account for overdispersion. It does not change the mean structure but adjusts the variance to be proportional to the mean.

Zero-Inflated Models:

- Zero-inflated Poisson (ZIP) or zero-inflated negative binomial (ZINB) models can be used if the data has an excess number of zeros, which can contribute to overdispersion.

Underdispersion

Underdispersion occurs when the variance of the count data is *less than the mean*. This is less common but can still occur. Here are some strategies to address underdispersion:

Generalized Poisson Regression:

- The generalized Poisson regression model allows for underdispersion by including a dispersion parameter that can be less than one.

Quasi-Poisson Regression:

- Similar to overdispersion, quasi-Poisson regression can be used to adjust the standard errors to account for underdispersion.

This article utilizes two datasets to demonstrate the modeling of count data. The first dataset, obtained from Kaggle, consists of credit card data (*CreditCard)*, which includes credit histories of a sample of applicants for a particular credit card. We will model the outcome variable, *active* (the number of active credit card accounts), in relation to other variables in the dataset. This number should be modeled as a Poisson model due to its large *overdispersion*.

The second dataset, also downloadable from Kaggle, is the *Smarket* data. Here, we will model the variable *Volume*, representing the count of total daily shares traded (in billions), using the percentage returns from the previous five days (Lag1 to Lag5). This dataset illustrates typical *underdispersion* in count data.

In this article, we have explored several regression models for count data modeling. Retaining or removing outliers and hyperparameter tuning will allow the reader to explore further.

Load the *CreditCard* dataset:

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder

cc_data = pd.read_csv('./Documents/AER_credit_card_data.csv', delimiter=',')

# convert categorical columns into numerical ones
label_encoder = LabelEncoder()
cc_data['owner'] = label_encoder.fit_transform(cc_data['owner'])
cc_data['selfemp'] = label_encoder.fit_transform(cc_data['selfemp'])
```

When working with Poisson Regression, scaling the independent variables may not be as critical as in other types of regression models, such as linear regression. This is because Poisson regression focuses on modeling count data and uses a logarithmic link function. Scaling the predictors would change the interpretation of these coefficients, making them less intuitive.

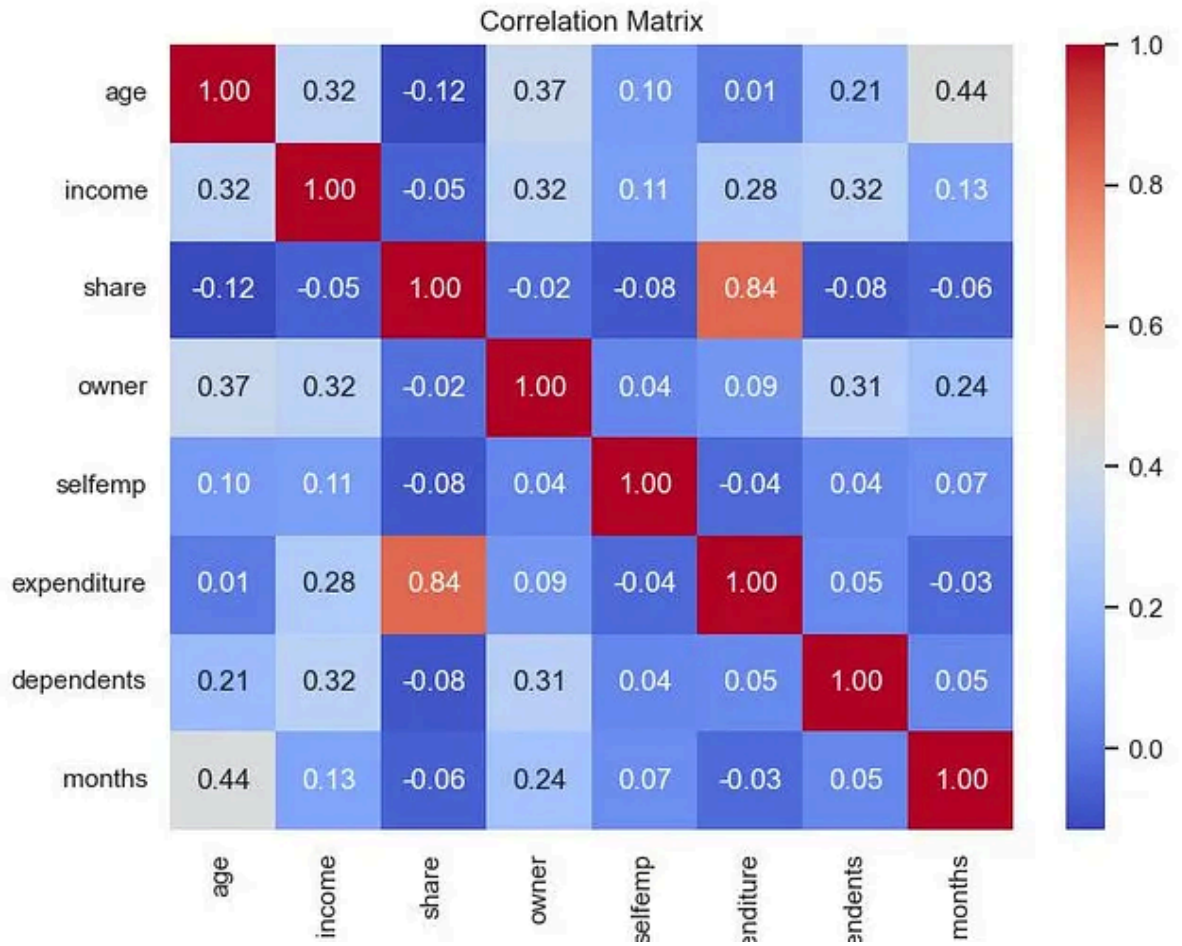
Check any multicollinearity on predictors:

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

sub_df = cc_data[['age','income','share','owner','selfemp', 'expenditure', 'depe

# Compute VIF for each predictor
vif_data = pd.DataFrame()
vif_data['Variable'] = sub_df.columns
vif_data['VIF'] = [variance_inflation_factor(sub_df.values, i) for i in range(su
```

```
print(vif_data.sort_values(by='VIF', ascending=False))
```

```
      Variable       VIF
0          age  8.045711
1       income  7.374535
5  expenditure  6.785199
2        share  6.058887
3        owner  2.273508
7       months  2.080822
6   dependents  1.948390
4      selfemp  1.098546
```

But “share” and “expenditure” shows high correlation. Here, we will drop “share” variable in our Poisson Regression analysis.



Fit Poisson regression, Quasi-Poisson, and Negative Binomial model on the training data:

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.model_selection import train_test_split

# Load the AER Credit Card dataset
data = cc_data.loc[:,"age":"active"]
formula = "active ~ age + income + expenditure + owner + selfemp + dependents +

# Split the data into training and testing sets
train_data, test_data = train_test_split(data, test_size=0.2, random_state=42)

# Fit Poisson regression model on the training data
cc_poisson_model = smf.glm(formula=formula, data=train_data, family=sm.families.
```

```
# Check for overdispersion
pearson_chi2 = cc_poisson_model.pearson_chi2
degree_freedom = cc_poisson_model.df_resid
dispersion = pearson_chi2 / degree_freedom

print(f"Dispersion parameter: {dispersion}")
# If dispersion > 1, data is overdispersed, < 1 implies underdispersion
print("\n\n")

# Fit Quasi-Poisson model
# The scale='X2' argument adjusts the standard errors of the estimates to accoun
cc_qp_model = smf.glm(formula=formula, data=train_data, family=sm.families.Poiss
#print(cc_qp_model.summary())

# Negative Binomial Model
cc_nb_model = smf.glm(formula=formula, data=train_data, family=sm.families.Negat
#print(cc_nb_model.summary())


# Print the model summary
print("Poisson regression model----------------------\n")
print(cc_poisson_model.summary())
print("\n")
print("Quasi-Poisson model---------------------------\n")
print(cc_qp_model.summary())
print("\n")
print("Negative Binomial Model-----------------------\n")
print(cc_nb_model.summary())
```

```
Dispersion parameter: 5.22133238472888



Poisson regression model----------------------

                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                 active   No. Observations:                 1055
Model:                            GLM   Df Residuals:                     1047
Model Family:                 Poisson   Df Model:                            7
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -4466.5
Date:                Sun, 09 Feb 2025   Deviance:                       5685.2
Time:                        12:39:34   Pearson chi2:                 5.47e+03
No. Iterations:                     5   Pseudo R-squ. (CS):             0.3732
Covariance Type:            nonrobust
```

```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept       1.4642      0.043     34.067      0.000       1.380       1.548
age             0.0039      0.001      2.825      0.005       0.001       0.007
income          0.0337      0.008      4.432      0.000       0.019       0.049
expenditure -1.934e-05   4.43e-05     -0.437      0.662      -0.000    6.74e-05
owner           0.4227      0.027     15.622      0.000       0.370       0.476
selfemp        -0.0648      0.046     -1.411      0.158      -0.155       0.025
dependents      0.0033      0.010      0.338      0.736      -0.016       0.022
months          0.0002      0.000      1.114      0.265      -0.000       0.001
==============================================================================


Quasi-Poisson model----------------------------

                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                  active   No. Observations:                 1055
Model:                             GLM   Df Residuals:                     1047
Model Family:                  Poisson   Df Model:                            7
Link Function:                     Log   Scale:                          5.2213
Method:                           IRLS   Log-Likelihood:                -855.43
Date:                Sun, 09 Feb 2025   Deviance:                       5685.2
Time:                        12:39:34   Pearson chi2:                 5.47e+03
No. Iterations:                     7   Pseudo R-squ. (CS):            0.08559
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept       1.4642      0.098     14.909      0.000       1.272       1.657
age             0.0039      0.003      1.236      0.216      -0.002       0.010
income          0.0337      0.017      1.940      0.052      -0.000       0.068
expenditure -1.934e-05      0.000     -0.191      0.848      -0.000       0.000
owner           0.4227      0.062      6.837      0.000       0.302       0.544
selfemp        -0.0648      0.105     -0.617      0.537      -0.270       0.141
dependents      0.0033      0.022      0.148      0.882      -0.040       0.047
months          0.0002      0.000      0.487      0.626      -0.001       0.001
==============================================================================


Negative Binomial Model-----------------------

                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                  active   No. Observations:                 1055
Model:                             GLM   Df Residuals:                     1047
Model Family:        NegativeBinomial   Df Model:                            7
Link Function:                     Log   Scale:                          1.0000
Method:                           IRLS   Log-Likelihood:                -3132.7
```

```
Date:                Sun, 09 Feb 2025   Deviance:                       1126.0
Time:                        12:39:34   Pearson chi2:                     735.
No. Iterations:                 7       Pseudo R-squ. (CS):           0.05868
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      1.4103      0.122     11.593      0.000       1.172       1.649
age            0.0052      0.004      1.320      0.187      -0.002       0.013
income         0.0375      0.023      1.623      0.105      -0.008       0.083
expenditure -1.365e-05      0.000     -0.105      0.917      -0.000       0.000
owner          0.4198      0.076      5.498      0.000       0.270       0.569
selfemp       -0.0968      0.131     -0.738      0.460      -0.354       0.160
dependents    -0.0060      0.029     -0.210      0.834      -0.062       0.050
months         0.0004      0.001      0.672      0.501      -0.001       0.001
==============================================================================
```

Fit the Generalized Poisson Regression model for overdispersion on the training data:

In *statsmodels*, it does not directly provide a *deviance* attribute for Generalized Poisson. we need to calculate the deviance manually using the log-likelihood of the fitted model and the log-likelihood of the saturated model.

Deviance is given by:

$$\text{Deviance} = -2\left(\text{llf}_{\text{model}} - \text{llf}_{\text{saturated}}\right)$$

$$\text{llf}_{\text{saturated}} = \sum_{i=1}^{n} \left(y_i \log(y_i) - y_i - \log(y_i!)\right)$$

```
from statsmodels.discrete.discrete_model import GeneralizedPoisson
from scipy.special import gammaln
```

```
Y = train_data["active"]
X = train_data[['age' , 'income' , 'expenditure' , 'owner' ,'selfemp' , 'depende

# Add a constant to the predictors
X = sm.add_constant(X)

# Fit the Generalized Poisson Regression model for overdispersion GP-2 (p=2)
# default optimization method (usually Newton-Raphson or BFGS) failed to converg
# 'nm' refers to the Nelder-Mead method, which is more robust but slower.

cc_gp_model = GeneralizedPoisson(endog = Y , exog = X, p=2).fit(method='nm')

# Print model summary
print(cc_gp_model.summary())


# Log-likelihood of the saturated model
def log_likelihood_saturated(y):
    return np.sum(y * np.log(y + 1e-10) - y - gammaln(y + 1))

llf_model = cc_gp_model.llf
llf_saturated = log_likelihood_saturated(Y)

# Calculate deviance
cc_gp_model_deviance = 2 * (llf_saturated - llf_model)
print(f"Deviance: {cc_gp_model_deviance}")
```

```
                    GeneralizedPoisson Regression Results
==============================================================================
Dep. Variable:                 active   No. Observations:                 1055
Model:             GeneralizedPoisson   Df Residuals:                     1047
Method:                           MLE   Df Model:                            7
Date:                Sun, 09 Feb 2025   Pseudo R-squ.:                0.006238
Time:                        12:48:59   Log-Likelihood:                -3184.4
converged:                      False   LL-Null:                       -3204.4
Covariance Type:            nonrobust   LLR p-value:                 1.270e-06
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          1.5280      0.111     13.795      0.000       1.311       1.745
age            0.0042      0.004      1.179      0.238      -0.003       0.011
income         0.0281      0.022      1.282      0.200      -0.015       0.071
expenditure -1.511e-05      0.000     -0.132      0.895      -0.000       0.000
owner          0.3427      0.066      5.172      0.000       0.213       0.473
selfemp       -0.0592      0.115     -0.517      0.605      -0.284       0.165
```

```
dependents      0.0035      0.026      0.135      0.892      -0.047       0.054
months          0.0002      0.001      0.397      0.691      -0.001       0.001
alpha           0.2032      0.009     23.234      0.000       0.186       0.220
==============================================================================

Deviance: 3121.088937742721
```

Among the 1,319 customers, there are 219 customers who have *zero* active credit cards, which is 16.6% of the total customers.

Fit the Zero-Inflated Poisson (ZIP) regression model on the training data:

```
#count occurrence of each value in 'active' column
counts = cc_data['active'].value_counts()

#count occurrence of each value in 'active' column as percentage of total
percs = cc_data['active'].value_counts(normalize=True)

#concatenate results into one DataFrame
count_active= pd.concat([counts,percs], axis=1, keys=['count', 'percentage'])
count_active['percentage'] = count_active['percentage'].apply(lambda x: f'{x * 1
count_active.loc[[0]]
```

```
       count percentage
active
0        219     16.60%
```

```
from statsmodels.discrete.count_model import ZeroInflatedPoisson
from scipy.special import gammaln

Y = train_data["active"]
X = train_data[['age' , 'income' , 'expenditure' , 'owner' ,'selfemp' , 'depende

# Add a constant to the predictors (if not already included)
```

```
X = sm.add_constant(X)

# Fit a Zero-Inflated Poisson (ZIP) regression model
cc_zip_model = ZeroInflatedPoisson(endog = Y , exog = X, exog_infl=X).fit()

# Print model summaries
print(cc_zip_model.summary())


# Log-likelihood of the saturated model
def log_likelihood_saturated(y):
    return np.sum(y * np.log(y + 1e-10) - y - gammaln(y + 1))

llf_model = cc_zip_model.llf
llf_saturated = log_likelihood_saturated(Y)

# Calculate deviance
zip_deviance = 2 * (llf_saturated - llf_model)
print(f"Deviance: {zip_deviance}")
```

```
         Current function value: 3.434414
         Iterations: 35
         Function evaluations: 64
         Gradient evaluations: 64
                     ZeroInflatedPoisson Regression Results
==============================================================================
Dep. Variable:                   active   No. Observations:                 1055
Model:             ZeroInflatedPoisson   Df Residuals:                     1047
Method:                            MLE   Df Model:                            7
Date:                 Sun, 09 Feb 2025   Pseudo R-squ.:                 0.05673
Time:                         09:55:40   Log-Likelihood:                -3623.3
converged:                       False   LL-Null:                       -3841.2
Covariance Type:             nonrobust   LLR p-value:                 4.927e-90
==============================================================================
                         coef    std err          z      P>|z|      [0.025
------------------------------------------------------------------------------
inflate_const          -1.2172      0.324     -3.754      0.000      -1.853
inflate_age             0.0012      0.011      0.116      0.907      -0.019
inflate_income          0.1482      0.059      2.499      0.012       0.032
inflate_expenditure    -0.0004      0.000     -1.200      0.230      -0.001
inflate_owner          -0.9549      0.214     -4.465      0.000      -1.374
inflate_selfemp        -0.4143      0.399     -1.039      0.299      -1.196
inflate_dependents      0.0888      0.073      1.214      0.225      -0.055
inflate_months         -0.0133      0.003     -4.886      0.000      -0.019
const                   1.6328      0.043     37.838      0.000       1.548
age                     0.0060      0.001      4.398      0.000       0.003
```

```
income                0.0549      0.008      7.056      0.000       0.040
expenditure        -6.76e-05   4.48e-05     -1.510      0.131      -0.000    2
owner                 0.2984      0.027     11.142      0.000       0.246
selfemp              -0.1559      0.046     -3.411      0.001      -0.245
dependents            0.0116      0.010      1.161      0.245      -0.008
months               -0.0008      0.000     -4.201      0.000      -0.001
==============================================================================

Deviance: 5402.497082374011
```

Summaries of all models:

```
metric_data = {
    "Model": ["Poisson", "Quasi-Poisson", "Negative Binomial", "Generalized Pois
    "AIC": [cc_poisson_model.aic, cc_qp_model.aic, cc_nb_model.aic, cc_gp_model.
    "BIC": [cc_poisson_model.bic, cc_qp_model.bic, cc_nb_model.bic, cc_gp_model.
    "Deviance": [cc_poisson_model.deviance, cc_qp_model.deviance, cc_nb_model.de
    "IIF" :  [cc_poisson_model.llf, cc_qp_model.llf, cc_nb_model.llf, cc_gp_mode
}

# Convert to dataframe
metric_df = pd.DataFrame(metric_data)

print(metric_df)
```

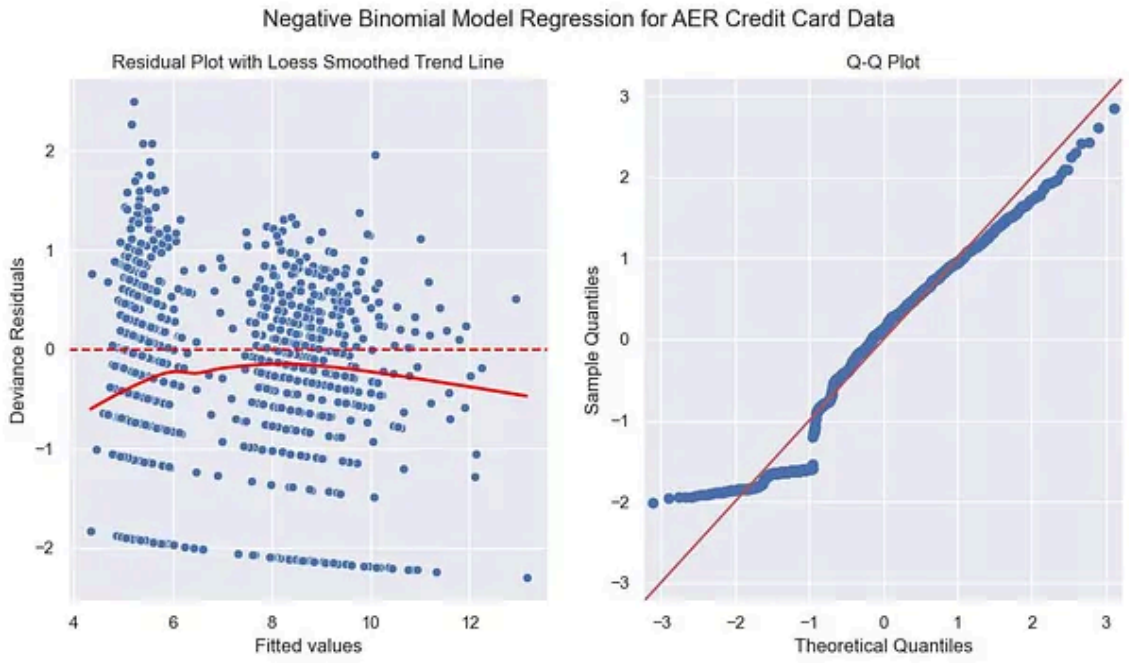
```
                  Model          AIC          BIC     Deviance          IIF
0               Poisson  8948.990352 -1603.294687  5685.182273 -4466.495176
1         Quasi-Poisson  1726.864142 -1603.294687  5685.182273  -855.432071
2     Negative Binomial  6281.413506 -6162.456297  1126.020663 -3132.706753
3   Generalized Poisson  6386.897016  6431.548681  3121.088938 -3184.448508
4  Zero Inflated Poisson  7278.614358  7357.995095  5402.497081 -3623.307179
```

Conclusion:

Since overdispersion is present, the Negative Binomial model might still be

preferred model. It has lowest BIC and deviance values.

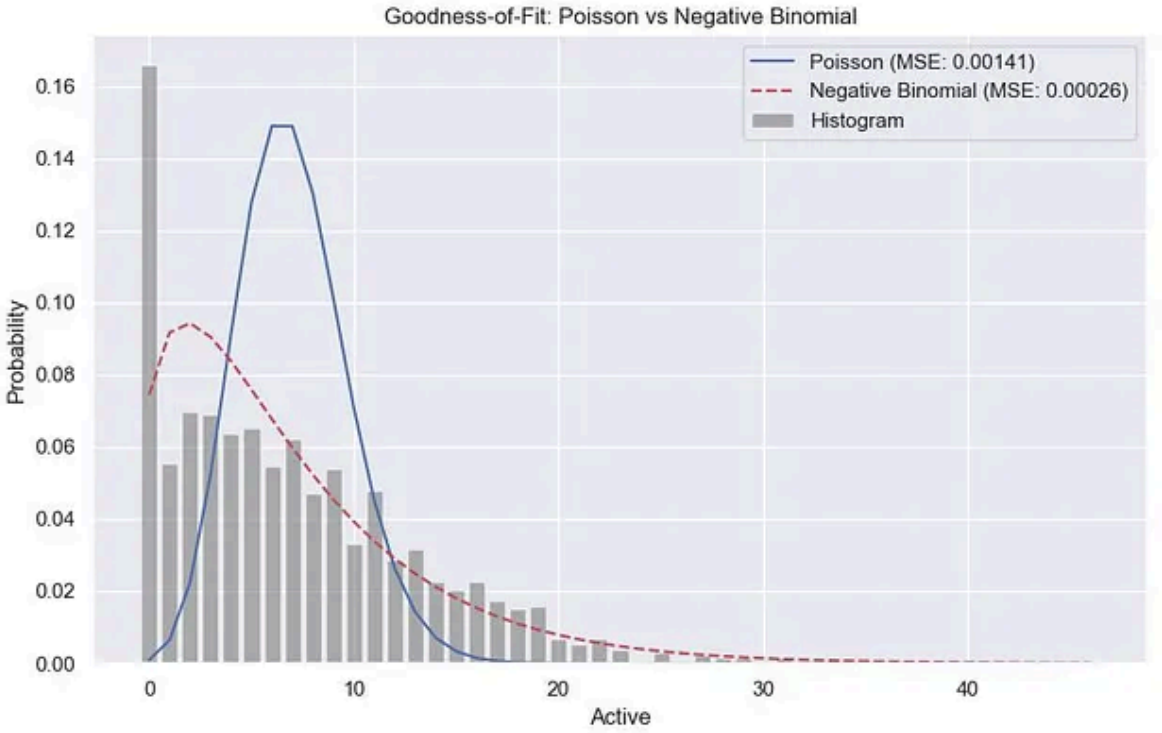
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm

# Deviance residuals
deviance_residuals = cc_nb_model.resid_deviance

# Plotting
fig, ax = plt.subplots(1, 2, figsize=(12, 6))
# Add an overall title
plt.suptitle('Negative Binomial Model Regression for AER Credit Card Data')

# Residual plot
sns.scatterplot(x=cc_nb_model.fittedvalues, y=deviance_residuals, ax=ax[0])

# Add a smoothed trend line
try:
    loess = sm.nonparametric.lowess(deviance_residuals, cc_nb_model.fittedvalues
    ax[0].plot(loess[:, 0], loess[:, 1], color='red', linewidth=2)
except Exception as e:
    print(f"Could not create LOWESS line: {e}")

ax[0].axhline(0, color='red', linestyle='--')
ax[0].set_xlabel('Fitted values')
ax[0].set_ylabel('Deviance Residuals')
ax[0].set_title('Residual Plot with Loess Smoothed Trend Line')

# Q-Q Plot
sm.qqplot(deviance_residuals, line='45', fit=True, ax=ax[1])
ax[1].set_xlabel('Theoretical Quantiles')
ax[1].set_ylabel('Sample Quantiles')
ax[1].set_title('Q-Q Plot')

#plt.tight_layout()

plt.show()
```



> *A good residual vs fitted plot has three characteristics [3]:*
>
> *The residuals "bounce randomly" around the 0 line. This suggests that the assumption that the relationship is linear is reasonable.*
>
> *The residuals roughly form a "horizontal band" around the 0 line. This suggests that the variances of the error terms are equal.*
>
> *No one residual "stands out" from the basic random pattern of residuals. This suggests that there are no outliers.*

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import poisson, nbinom
from sklearn.metrics import mean_squared_error

# Extract the 'active' column
active = cc_data['active']

# Calculate histogram for visualization
```

```
values, bins = np.histogram(active, bins=range(active.min(), active.max() + 2),
bin_centers = np.arange(active.min(), active.max() + 1)  # Ensure discrete value

# Fit Poisson distribution
poisson_lambda = np.mean(active)
poisson_pmf = poisson.pmf(bin_centers, mu=poisson_lambda)

# Fit Negative Binomial distribution
mean_active = np.mean(active)
var_active = np.var(active)
if var_active > mean_active:  # Check for overdispersion
    nb_p = mean_active / var_active
    nb_r = mean_active**2 / (var_active - mean_active)  # r parameter (successes
    negative_binomial_pmf = nbinom.pmf(bin_centers, n=nb_r, p=nb_p)
else:
    nb_p = 1  # In case there's no overdispersion, NB reduces to Poisson
    nb_r = 1
    negative_binomial_pmf = poisson.pmf(bin_centers, mu=poisson_lambda)

# Calculate goodness-of-fit
poisson_mse = mean_squared_error(values, poisson_pmf)
nb_mse = mean_squared_error(values, negative_binomial_pmf)

# Print goodness-of-fit metrics
print("Goodness-of-Fit Metrics:")
print(f"Poisson MSE: {poisson_mse:.5f}")
print(f"Negative Binomial MSE: {nb_mse:.5f}")

# Plot histogram and fitted distributions
plt.figure(figsize=(10, 6))
plt.bar(bin_centers, values, width=0.8, alpha=0.6, color='gray', label='Histogra
plt.plot(bin_centers, poisson_pmf, 'b-', label=f'Poisson (MSE: {poisson_mse:.5f}
plt.plot(bin_centers, negative_binomial_pmf, 'r--', label=f'Negative Binomial (M
plt.xlabel('Active')
plt.ylabel('Probability')
plt.title('Goodness-of-Fit: Poisson vs Negative Binomial')
plt.legend()

plt.show()
```



Load the *Smarket* dataset:

```
import numpy as np
import pandas as pd

sm_data = pd.read_csv('./Documents/Smarket.csv', delimiter=',')
```

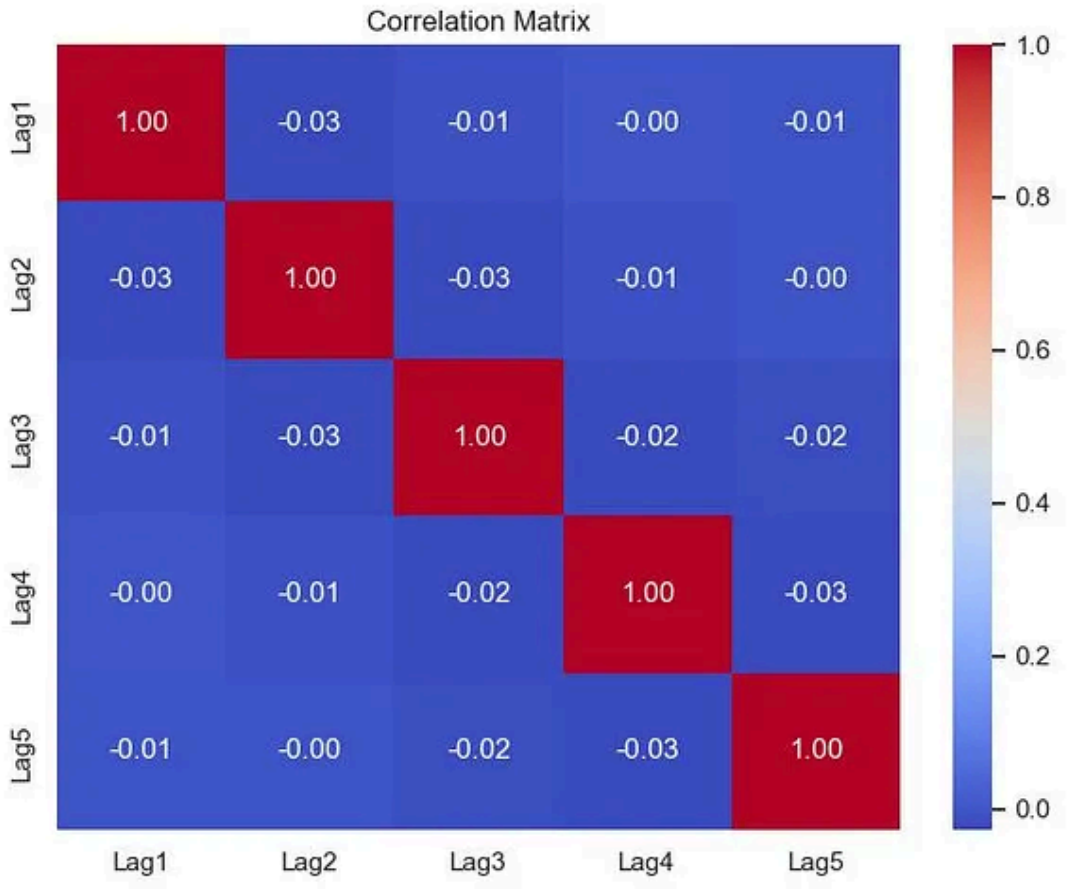
Check any multicollinearity on predictors:

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```
sub_df2 = sm_data[['Lag1','Lag2','Lag3','Lag4','Lag5']]

# Compute VIF for each predictor
vif_data = pd.DataFrame()
vif_data['Variable'] = sub_df2.columns
vif_data['VIF'] = [variance_inflation_factor(sub_df2.values, i) for i in range(s

print(vif_data.sort_values(by='VIF', ascending=False))
```

```
  Variable       VIF
2     Lag3  1.001784
1     Lag2  1.001532
3     Lag4  1.001487
4     Lag5  1.001169
0     Lag1  1.000873
```

```
# Compute the correlation matrix
correlation_matrix = sub_df2.corr()

# Display the correlation matrix
print(correlation_matrix)

# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", cbar=Tru
plt.title('Correlation Matrix')

plt.show()
```



Fit Poisson regression and Quasi-Poisson model on the training data:

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.model_selection import train_test_split

# Load the AER Credit Card dataset
data = sm_data.loc[:,"Lag1":"Volume"]
formula = "Volume ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5"

# Split the data into training and testing sets
train_data, test_data = train_test_split(data, test_size=0.2, random_state=42)

# Fit Poisson regression model on the training data
sm_poisson_model = smf.glm(formula=formula, data=train_data, family=sm.families.

# Check for overdispersion
pearson_chi2 = sm_poisson_model.pearson_chi2
```

```
degree_freedom = sm_poisson_model.df_resid
dispersion = pearson_chi2 / degree_freedom

print(f"Dispersion parameter: {dispersion}")
# If dispersion > 1, data is overdispersed, < 1 implies underdispersion
print("\n\n")

# Fit Quasi-Poisson model
# The scale='X2' argument adjusts the standard errors of the estimates to accoun
sm_qp_model = smf.glm(formula=formula, data=train_data, family=sm.families.Poiss
#print(cc_qp_model.summary())

# Print the model summary
print("Poisson regression model----------------------\n")
print(sm_poisson_model.summary())
print("\n")
print("Quasi-Poisson model---------------------------\n")
print(sm_qp_model.summary())
```

```
Dispersion parameter: 0.08705718736182032



Poisson regression model----------------------

                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                 Volume   No. Observations:                 1000
Model:                            GLM   Df Residuals:                      994
Model Family:                 Poisson   Df Model:                            5
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -1202.4
Date:                Sun, 09 Feb 2025   Deviance:                       84.563
Time:                        13:12:32   Pearson chi2:                     86.5
No. Iterations:                     4   Pseudo R-squ. (CS):          0.0007017
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      0.3946      0.026     15.189      0.000       0.344       0.446
Lag1           0.0055      0.023      0.234      0.815      -0.040       0.051
Lag2          -0.0065      0.023     -0.279      0.780      -0.052       0.039
Lag3          -0.0113      0.023     -0.489      0.625      -0.057       0.034
Lag4          -0.0102      0.023     -0.451      0.652      -0.055       0.034
Lag5          -0.0092      0.023     -0.406      0.685      -0.054       0.035
==============================================================================
```

```
Quasi-Poisson model----------------------------

                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                 Volume   No. Observations:                 1000
Model:                            GLM   Df Residuals:                      994
Model Family:                 Poisson   Df Model:                            5
Link Function:                    Log   Scale:                         0.087057
Method:                          IRLS   Log-Likelihood:                -13812.
Date:                Sun, 09 Feb 2025   Deviance:                       84.563
Time:                        13:12:32   Pearson chi2:                     86.5
No. Iterations:                     6   Pseudo R-squ. (CS):           0.008031
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      0.3946      0.008     51.480      0.000       0.380       0.410
Lag1           0.0055      0.007      0.794      0.427      -0.008       0.019
Lag2          -0.0065      0.007     -0.945      0.345      -0.020       0.007
Lag3          -0.0113      0.007     -1.656      0.098      -0.025       0.002
Lag4          -0.0102      0.007     -1.527      0.127      -0.023       0.003
Lag5          -0.0092      0.007     -1.376      0.169      -0.022       0.004
==============================================================================
```

Fit the Generalized Poisson Regression model for underdispersion on the training data:

```
from statsmodels.discrete.discrete_model import GeneralizedPoisson
from scipy.special import gammaln

Y = train_data["Volume"]
X = train_data[['Lag1','Lag2','Lag3','Lag4','Lag5']]

# Add a constant to the predictors
X = sm.add_constant(X)

# Fit the Generalized Poisson Regression model for underdispersion GP-1 (p=1)
# default optimization method (usually Newton-Raphson or BFGS)
sm_gp_model = GeneralizedPoisson(endog = Y , exog = X, p=1).fit()
```

```
# Print model summary
print(sm_gp_model.summary())

# Log-likelihood of the saturated model
def log_likelihood_saturated(y):
    return np.sum(y * np.log(y + 1e-10) - y - gammaln(y + 1))

llf_model = sm_gp_model.llf
llf_saturated = log_likelihood_saturated(Y)

# Calculate deviance
sm_gp_model_deviance = 2 * (llf_saturated - llf_model)
print(f"Deviance: {sm_gp_model_deviance}")
```

```
Optimization terminated successfully.
         Current function value: 0.584446
         Iterations: 19
         Function evaluations: 41
         Gradient evaluations: 31
                 GeneralizedPoisson Regression Results
==============================================================================
Dep. Variable:                  Volume   No. Observations:                 1000
Model:             GeneralizedPoisson   Df Residuals:                      994
Method:                           MLE   Df Model:                            5
Date:                Sun, 09 Feb 2025   Pseudo R-squ.:                 0.04417
Time:                        13:18:34   Log-Likelihood:                -584.45
converged:                       True   LL-Null:                       -611.45
Covariance Type:            nonrobust   LLR p-value:                 2.085e-10
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.3949      0.012     33.950      0.000       0.372       0.418
Lag1           0.0050      0.005      0.978      0.328      -0.005       0.015
Lag2          -0.0209      0.005     -4.280      0.000      -0.030      -0.011
Lag3          -0.0282      0.006     -4.480      0.000      -0.041      -0.016
Lag4          -0.0080      0.007     -1.158      0.247      -0.021       0.006
Lag5          -0.0054      0.008     -0.674      0.500      -0.021       0.010
alpha         -0.5520      0.007    -78.348      0.000      -0.566      -0.538
==============================================================================
Deviance: -1151.3965106773949
```

```
metric_data = {
    "Model": ["Poisson", "Quasi-Poisson", "Generalized Poisson"],
    "AIC": [sm_poisson_model.aic, sm_qp_model.aic, sm_gp_model.aic],
    "BIC": [sm_poisson_model.bic, sm_qp_model.bic,  sm_gp_model.bic],
    "Deviance": [sm_poisson_model.deviance, sm_qp_model.deviance,  sm_gp_model_d
    "IIF" :  [sm_poisson_model.llf, sm_qp_model.llf, sm_gp_model.llf]
}

# Convert to dataframe
metric_df = pd.DataFrame(metric_data)

print(metric_df)
```

	Model	AIC	BIC	Deviance	IIF
0	Poisson	2416.851299	-6781.746167	84.562581	-1202.425649
1	Quasi-Poisson	27635.811102	-6781.746167	84.562581	-13811.905551
2	Generalized Poisson	1182.892207	1217.246494	-1151.396511	-584.446104

Conclusion:
The Generalized Poisson model is the best fit overall, as it has the lowest AIC, lowest deviance, and highest log-likelihood. These metrics consistently favor the Generalized Poisson model.

While the Poisson and Quasi-Poisson models have a lower BIC, the BIC result is likely an outlier or anomaly, as the other metrics strongly favor the Generalized Poisson model.

From above model, we notice that there is no statistical significance for all the covariates of Lag1 to Lag5 indicating that the percentage of the previous 5 days are not statistically significant predictors for the Volume and other predictors should be included in prediction model.

Reference:
[1] You Can Build Any Linear Model If You Learn Just One Thing About Them
[2] Financial Data Analytics with R - Monte-Carlo Validation, Jenny K. Chen, (2024, CRC Press)
[3] https://stats.stackexchange.com/questions/253035/trying-to-understand-the-fitted-vs-residual-plot

Thank you for reading.

Written by Simon Leung

229 followers · 4 following

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.

Edit profile